Finding SNAP Payment Errors Faster: Synthetic Data, Language Models and Triage Rankers

October 9, 2026 · Executive whitepaper · Cory Trimm

Executive summary

States lose money and federal standing when SNAP benefits are paid in the wrong amount. Quality control (QC) reviewers can only examine a small sample of cases, so the cases they choose matter. This project asked whether machine learning could help them choose better, and whether a small, specialized language model could act as a copilot for QC reviewers.

Over ten days and more than 100 experiments (including pilots for six states), using five years of federal QC data (FY2020–2024, about 167,000 reviewed cases), we reached three findings:

1. **A ranking tool is ready to pilot.** For each of six states (Pennsylvania, New York, Texas, South Dakota, Utah and Idaho), we built a model that ranks cases by expected error dollars. On cases it never saw, reviewing the top 20% of its queue finds **2.6 to 3.5 times the error dollars of random selection**. In New York that is about $199 of errors found per review hour, against $56 at random. It uses only information the agency has before review, and its dollar forecasts land within about 13% of actual in four of the six states.
2. **Realistic synthetic cases can teach a model real skills.** We built a generator that produces fake SNAP cases with known errors, calibrated to the federal data. Models trained only on these synthetic cases predict errors in real cases well above chance.
3. **A small language model can learn part of the QC task from synthetic data alone.** An open model of about 4 billion parameters, trained only on synthetic cases, went from guessing to passing all three of our quality checks on 1,200 fresh real QC cases, under rules fixed before the cases were drawn. It outperformed a conventional classifier trained on the same data and, on our original test set, an untrained model eight times its size. It spots errors, their direction and their location well above chance, but can barely tell the type of mistake and can't tell whose mistake it was.

Recommendation: pilot the ranking tool in one state (Pennsylvania or New York, where it is strongest and most accurate). Keep the language model as a research track until it adds something the ranking tool can't, such as written explanations for reviewers.

The problem

Every year, each state's QC staff review a random sample of SNAP cases and record whether the benefit was correct, overpaid or underpaid, by how much, and why. That sample sets the state's official payment error rate, which carries federal financial consequences.

Beyond the official sample, states run their own internal reviews to catch errors early. Reviewer time is the scarce resource. If reviewers could start with the cases most likely to hold large errors, the same staff would find more dollars of error and fix the causes sooner.

The federal government publishes the QC review data each year with personal details removed. It shows each sampled household's reported income, deductions, household makeup and benefit, plus what the reviewer found. That is enough to learn from, but with limits: up to about 45,000 cases a year nationally, only about 470–820 a year per state, and no written reasoning.

Approach

The project ran in three tracks, each building on the one before.

1. A synthetic case generator

Real QC data is too small to train a language model and can't be shared freely. So we built a generator that creates realistic fake households, computes their correct benefit using the federal benefit rules, and then injects errors: wrong income, missed deductions, household-composition mistakes and timing errors. Each error comes with the exact right answer.

We tuned the generator, experiment by experiment, until its output matched the real data on what matters: how often errors occur, overpayment versus underpayment, which part of the case is wrong, why, and how many dollars are involved. The overall error mix now matches the real data to within about 2%, and the dollar error rate to within about one percentage point. We also built versions tuned to individual states.

The test that matters is **transfer**: train a model only on synthetic cases, then test it on real ones. Across many experiments, synthetic-trained classifiers beat chance on real cases every time, reaching about half to two-thirds of what a model trained on real data achieves. Adding more synthetic data stopped helping past about 10,000 cases; making the cases more realistic was what moved results.

2. A small language model for QC review

The original goal was a small, specialized language model that reads a case the way a reviewer does. To measure that, we built an evaluation of 1,400 real QC cases written as reviewer-style case summaries, with up to six questions per case (the detail questions apply only to cases with errors):

Question	What the model must say
T1	Is there an error?
T2	Correct, overpaid, or underpaid?
T3	Which part of the case is wrong (income, shelter, household size…)?
T4	What kind of mistake (e.g. unreported wages)?
T5	Whose mistake (agency or client)?
T6	How many dollars?

We set three pass/fail checks before training anything. One was revised after early results, so the final test was rerun on fresh cases under rules committed in advance (see Results). The trained model must (1) beat the untrained model by a clear statistical margin, (2) beat it on at least four of the five category questions, and (3) identify the error's location (T3) at least as well as a conventional classifier trained on the same synthetic data. The dollar question proved nearly unanswerable from case text for every model and was dropped from scoring.

We fine-tuned Google's Gemma 4 E4B (about 4 billion parameters) on a Mac Studio. The method that worked was supervised fine-tuning: showing the model synthetic cases with correct answers, and stopping training early. Reinforcement learning on its own taught the model only to guess the most common answers. With supervised fine-tuning first, it learned real skill.

3. Triage rankers for individual states

In parallel, we asked a more practical question: if the goal is finding error dollars, what is the simplest tool that does it? The answer is a conventional machine-learning model (gradient-boosted decision trees, a standard method for spreadsheet-style data) with two parts. One part estimates the chance a case has an error. The other estimates how many dollars the error would be. Multiplying them gives expected error dollars, and cases are reviewed in that order.

We trained these rankers on each state's own QC records and tuned them with standard cross-validation. Each was tested three ways: on held-out cases from the same years, on a later time period (train on 2020–22, test on 2023–24), and against random selection.

Results

The language model learned the task from synthetic data alone

Scores on 1,200 fresh real QC cases that no model had been tested on, under pass/fail rules written down and committed before the cases were drawn. "Skill" is the average score above what lucky guessing would get, where 0% means no better than guessing:

Model	Overall score	Skill above chance	Finds error location (T3)	Passes all checks?
Gemma 4, 4B, untrained	0.5%	0.5%	1%	—
Conventional classifier, same synthetic data	24%	15%	18%	—
Gemma 4, 4B, trained on synthetic cases	**33%**	**28%**	**24%**	**Yes**

The bar for the location check was 21%, fixed in advance. The trained model beat the untrained one on all five questions (overall gain +33 points, 95% interval +31 to +35), and passed on the most recent fiscal year on its own (overall 38%). On the original test set it had scored 30%, so the fresh result is slightly higher. On that original set, an untrained Gemma 4 eight times larger scored 13%.

It is still well short of a human reviewer. It is clearly better than chance at spotting an error, its direction and its location. It is only slightly better than chance at naming the type of mistake (T4), and no better at saying whose mistake it was (T5).

The rankers find two to three and a half times more error dollars

Results on held-out cases each model never saw, reviewing the top 20% of its queue. Reviews are assumed to take 30 minutes each.

State	Error dollars found vs. random	Found per review hour (random)	Top 10% of queue vs. random	Dollar forecast vs. actual (top 20%)
Pennsylvania	3.5×	$165 ($47)	5.4×	4% under
New York	3.5×	$199 ($56)	5.2×	7% under
Texas	2.7×	$144 ($54)	3.6×	11% under
South Dakota	2.7×	$60 ($22)	2.9×	13% under
Utah	2.6×	$69 ($26)	3.7×	11% over
Idaho	2.8×	$62 ($22)	4.1×	25% over

These replace the figures in the earlier state pilot whitepapers (for example, $67 per hour in Pennsylvania). Those rankers were trained on synthetic cases, to test the synthetic data. These are trained on each state's real QC records, which is what a deployed tool would do, and they find about two and a half times as many dollars per review hour.

Three things make these numbers usable:

- **They hold over time.** Trained on 2020–22 and tested on 2023–24, the rankers kept most of their edge (2.3–3.3× random). A once-a-year refresh is enough.
- **The dollar forecasts are calibrated.** The raw model underestimated error dollars by 36–49%. A standard calibration step, fit on predictions for cases each model hadn't trained on, corrected that, so a state can say "the top fifth of this month's queue likely holds about $X of errors" with a ±15–20% band.

- **They use only pre-review information.** An audit found that earlier versions used four fields that contain the QC reviewer's corrected values, which a real tool wouldn't have until after review. Rebuilt without them, the rankers did as well or better in every state (Idaho within 0.03×). The useful signals are what the agency already records: reported income, deductions, housing costs, household size and the benefit issued.

What else we learned

- **Each state's errors look different.** Pennsylvania's errors cluster in how deductions are calculated, especially shelter costs; New York's are mostly about timing. That's why each state needs its own model.
- **Some states don't record dollar amounts per error.** In 22 of 53 states, most error cases leave that field blank (Texas almost all of them), while Pennsylvania and New York fill it in. Tools that explain *where* the error was depend on states that record it.
- **Fairness can be built in cheaply.** Ranking purely by dollars gives elderly and disabled households less review than their share of errors. Adding a minimum review share per household type raised their coverage 1.4–1.9 times with no measurable loss in dollars found (Pennsylvania and New York).
- **Ranking doesn't need a language model.** The conventional ranker already does the job well, trained on each state's own records, and costs almost nothing to run. The language model's potential lies in reading and explaining cases.

Methodology notes

- **Data:** federal SNAP QC public-use files, FY2020–2024, about 167,000 reviewed cases nationally. State rankers used each state's own 2,365–4,101 cases. Each state set aside about a quarter of its cases for final testing.
- **Synthetic generator:** a household sampler, the federal benefit formula, and an error injector whose rates come from the real data. Calibration and transfer were checked after every change, with each experiment recorded as hypothesis, method, result, verdict.
- **Language model:** Gemma 4 E4B (about 4 billion parameters), fine-tuned with LoRA adapters on synthetic cases only (supervised fine-tuning, about 400 steps).

Reinforcement learning (GRPO) was tested and set aside. Training ran locally on a Mac Studio; a Google Colab version was also built. Scoring used macro-averaged F1 relative to guessing, with bootstrap confidence intervals and chance-corrected agreement (Cohen's kappa).

- **Rankers:** two-part gradient-boosted models (scikit-learn), probability of error times expected dollars, tuned by nested cross-validation. Expected dollars are calibrated with isotonic regression on out-of-fold predictions. They are evaluated on held-out cases and on a later time period.
- **Rigor:** pass/fail criteria were set before training, the one later revision was confirmed on fresh cases under rules committed before they were drawn, the real test cases were never used for training or tuning, and every result, including failures, is recorded in the project's experiment log.

Limitations

- **Public data only.** The models were built from the federal public-use files, which omit some case detail and identifiers. A state's own case-management data would likely do better, and would need a short adaptation step.
- **Small per-state samples.** With a few thousand cases per state, results for smaller states (South Dakota, Utah, Idaho) carry wider uncertainty, which shows in their dollar forecasts.
- **The language model is a research result, not a product.** It passed a pre-registered test on fresh cases, but only on one kind of task (classifying public-use case records), and it is well short of a human reviewer on the details of an error.
- **The rankers prioritize; they don't decide.** They rank cases for human review. They don't make determinations about any household.

Recommendations

1. **Pilot the ranker in one state.** Start with Pennsylvania or New York, where results and dollar forecasts are strongest. Score a current month's cases, have reviewers work the top of the queue, and compare the dollars found against the forecast.
2. **Refresh annually** with the latest QC year.

3. **Include the fairness floor** so elderly and disabled households keep a proportionate share of reviews.
4. **Continue the language model as research** aimed at what the ranker can't do: plain-language explanations of why a case looks risky.

Project timeline

When	Milestone
Sept 28–29	Project started. Synthetic generator built and calibrated over ~45 experiments, and the language-model evaluation designed; first proof that synthetic-trained models predict real errors. Pennsylvania and New York pilots, each with its own whitepaper.
Oct 2–3	Fairness analysis: how dollar-focused ranking affects elderly, disabled and other household types, and a review floor that fixes it.
Oct 4	Pilots for Texas, South Dakota, Utah and Idaho, each with its own whitepaper. Language model evaluation run on two Gemma 4 models: the untrained small model scored near zero; the larger one only slightly better.
Oct 5–6	Small model trained on synthetic cases: reinforcement learning alone failed; supervised fine-tuning worked. A conventional classifier on the same data scored similarly.
Oct 7	Training data made consistent with how the evaluation labels cases; best language model passes all checks on the original test set. Triage rankers tuned for six states, dollar forecasts calibrated, rebuilt on pre-review data only, and packaged for a pilot.
Oct 9	Pre-registered confirmation on 1,200 fresh real cases: the language model passes all three checks (overall 33%, location 24% vs a 21% bar).